计算怎样成为证明

计算机显示“结果为正”，为什么还不能立即算作证明？在临界线上找到几百个零点，为什么还不能说这些高度以下的黎曼猜想已经验证？本文从一个能用笔算核对的例子出发，逐步增加难度，最后完成一次完整的计算证明：高度 600 以下的全部非平凡零点都在临界线上。每前进一步，只增加一个新的逻辑环节。所需程序完整收入附录。

1 计算与证明的关系

1.1 计算也是一种证明

常有一种说法：计算是数值运算，证明是逻辑推理。这个区分并不准确。

精确计算本身就是证明。等式 $641 \times 6700417 = 4294967297$ 可以从自然数的公理（Peano 公理与乘法的递归定义）一步步推出，它是一条定理；竖式乘法只是这一推导的简便写法。在 Lean 等证明助手中，这类等式正是用“把两边算出来比对”的方式完成证明的，内核检查的就是计算过程。反过来，证明中也常常含有计算：展开代数恒等式、求积分、做模幂运算，都是证明里的计算步骤。

真正的区别在于两者回答的问题不同：

- **计算**对给定的具体输入，按确定的步骤得出结果，回答“这一个情形是什么”；
- **证明**说明一个命题为什么成立，通常涉及无穷多个情形，回答“为什么一切情形都如此”。

二者的连接点是：一条定理把无穷多的可能归结为有限个检查，计算可靠地完成这些检查。例如第 5 节中，Turing 的判据说明为什么检查有限个符号和一个积分就足以推出“高度 600 以下全部零点在临界线上”；而上万次符号判定和区间求积，则是完成这些检查的计算。

需要特别小心的是近似计算。浮点运算每一步都可能舍入，屏幕上显示“0.00001”并不是一条逻辑推导，因为真值偏离多少并不知道。第 3 节的区间算术，正是把近似计算重新变成逻辑：每一步都保证“真值在这个区间内”，于是“下端点大于 0”可以严格推出“真值大于 0”。

1.2 四种结论

同样是“用计算机算出来的”，结论的种类不同，所需的论证也不同。本文按由易到难的顺序讨论四种：

结论的种类	本文的例子	比前一种多出的要求
算术事实	$2^{32}+1$ 不是素数（第 2 节）	只用整数运算，没有误差
数值不等式	$\sqrt{2} > 1.414$；Li 系数 $\lambda_1 > 0$（第 3 节）	必须把近似误差严格包住
存在性	临界线上 14.1347 与 14.1348 之间有零点（第 4 节）	还要用定理把“符号”转化为“零点”
完整性	高度 600 以下的零点全在临界线上（第 5 节）	还要证明没有遗漏，包括临界线外

最后一种就是有限高度的黎曼猜想。记

$$\mathrm{RH}(T):\quad \zeta(\beta+\mathrm{i}\gamma)=0,\quad 0<\beta<1,\quad 0<\gamma\le T\quad\Longrightarrow\quad \beta=\tfrac{1}{2}.$$

只写正纵坐标，是因为负纵坐标的零点由共轭对称性得到。

无论哪一种，计算要成为证明，都需要三方面配合：**论证**——一条数学定理，说明检查哪些有限事实就足以推出结论；**计算**——可靠地完成这些检查；**核查**——确认输入、范围和误差符合定理的条件，并且检查本身可以被他人重复。每节末尾用一个方框列出这三方面在该例中分别是什么。

2 算术事实：证书使检查变得容易

2.1 一个历史例子

Fermat 猜想所有形如 $F_n=2^{2^n}+1$ 的数都是素数；$F_0,\dots,F_4=3,5,17,257,65537$ 确实都是素数。1732 年 Euler 发现

$$F_5=2^{32}+1=4294967297=641\times 6700417,$$

从而否定了这一猜想。

这个等式谁都可以核对：做一次乘法就够了，不需要知道 Euler 是怎样找到 641 的。找因子很难——Euler 需要先论证 F_5 的素因子只能取 $64k+1$ 的形式，再逐个试除；检查却只需一次乘法。这一对比就是计算证明中最基本的结构：

程序交出有限的数据（证书），定理说明这些数据为什么足够，检查者只核对数据。

这里的“定理”只是乘法的定义：若 $n=ab$ 且 $1<a<n$，则 n 不是素数。搜索因子的程序可能写得很复杂，甚至含有错误，但这不影响结论：一个错误的候选因子在核对时就会被拒绝。

2.2 证明“是素数”需要另一种证书

上面的证书证明的是“不是素数”。要证明一个大数是素数，不能只给一个因子；但同样存在简短的证书。Lucas 型判据用一个底数 a 和 $n-1$ 的素因子分解作证书，检查只需若干次模幂运算；Pratt 证明了每个素数都有这样的简短证书 [1]。本书配套仓库“形式化证明”栏目中的例程 `Mersenne61.lean`，就是用 Lucas–Lehmer 计算证书在 Lean 中核验 $2^{61}-1$ 为素数。这些判据的证明需要群论，本文不展开；只需记住：证书的形式可以不同，“搜索难、检查易”的结构不变。

2.3 这里需要信任什么

整数运算没有舍入误差，但“没有舍入误差”不等于“程序必然无错”。证书的好处在于，检查程序可以写得很简单，甚至可以用独立实现重复检查；结论因此不依赖于搜索程序是否正确。

本节的分工 论证：$n=ab$，$1<a<n$ 推出 n 是合数。 计算：一次整数乘法。 核查：任何人可以重做这次乘法。

3 数值不等式：用区间包住误差

3.1 先看一个熟知的例子

要证明 $1.414 < \sqrt{2} < 1.415$，不必算出 $\sqrt{2}$ 的很多位，只需验证

$$1.414^2 = 1.999396 < 2 < 2.002225 = 1.415^2,$$

再用 $x \mapsto x^2$ 在正数上递增即可。这里的 1.414 与 1.415 按精确有理数理解，全部运算都是有理数运算，没有误差。承担证明作用的是两个端点，而不是 $\sqrt{2}$ 的近似值 1.41421... 显示了多少位。

这就是区间方法的基本思想：不写“$x \approx$ 某数”，而写

$$x \in [\ell, u].$$

要证明 $x > 0$，只需证明 $\ell > 0$。

3.2 计算值与真值必须分开

若屏幕显示 $x \approx 0.00001$，而误差可能达到 0.0001，则真值仍可能为负。即使两次提高精度后显示相同，也不能单凭这一现象排除共同的算法错误或未计入的截断误差。

区间算术把每个量表示为一个包含真值的区间，并保证每一步运算后区间仍然包含真值。最简单的规则是

$$[a,b] + [c,d] = [a+c, b+d], \qquad [a,b] - [c,d] = [a-d, b-c].$$

乘法取四个端点乘积的最小值和最大值；除法要求除数区间不含 0。机器不能精确表示端点时，下端点向下舍入，上端点向上舍入，使真值不被挤出区间。沿表达式逐步计算，最终区间也包含真值（对表达式的构造作归纳即可证明）。

对 log、Γ、ζ 等特殊函数，仅给普通浮点结果加一个小半径并不够：函数的计算方法本身必须同时控制舍入误差和级数截断误差。本文采用 FLINT 中的 Arb 球算术 [2]，由其函数实现提供这种包含保证。球 $[m \pm r]$ 就是区间 $[m-r, m+r]$ 的另一种写法。

包含关系不保证区间很窄。例如对同一个 $x \in [0,1]$，$x - x = 0$，但直接做区间减法得到 $[-1,1]$。因此，区间包含 0 只说明本次计算没有判定符号，不能据此断言真值为 0；这时应提高精度或改写算式。

3.3 例子：第一个 Li 系数为正

学习笔记第 10 篇给出

$$\lambda_1 = 1 + \frac{\gamma_E}{2} - \frac{1}{2}\log(4\pi),$$

其中 γ_E 为 Euler 常数。与 $\sqrt{2}$ 不同，γ_E 和 $\log(4\pi)$ 无法用有限次有理运算精确算出，这里才需要区间算术。Arb 的常数函数与对数函数给出包含真值的球，程序据此严格确认

$$0.57721 < \gamma_E < 0.57722, \qquad 2.53102 < \log(4\pi) < 2.53103.$$

其后的推导与 $\sqrt{2}$ 的例子相同，只是有理数运算：

$$\lambda_1 > 1 + 0.57721/2 - 2.53103/2 = 0.02309,$$

$$\lambda_1 < 1 + 0.57722/2 - 2.53102/2 = 0.02310.$$

所以 $\lambda_1 > 0$。结论的范围也要说清楚：这里只证明了一个 Li 系数为正，不能推出所有 Li 系数非负，更不能推出 RH。

本节的分工 论证：λ_1 的公式；区间运算保持包含关系。 计算：γ_E、$\log(4\pi)$ 的包含区间。 核查：端点比较是有理数不等式，可手算重做。

4 存在性：由符号变化得到零点

4.1 把复数的零点问题变成实数的符号问题

复数没有正负号，不能直接使用介值定理。为此引入 Hardy 函数

$$Z(t) = e^{\mathrm{i}\vartheta(t)}\zeta(\tfrac{1}{2} + \mathrm{i}t), \qquad t \in \mathbb{R},$$

其中相位 $\vartheta(t)$ 等于 $\Gamma(\frac{1}{4} + \frac{\mathrm{i}t}{2})$ 的辐角减去 $\frac{t}{2}\log\pi$，辐角从 $\vartheta(0) = 0$ 出发随 t 连续取值。由 ζ 的函数方程可以推出 $Z(t)$ 对实数 t 取实值（本书第 19 章）；又因 $e^{\mathrm{i}\vartheta(t)} \neq 0$，$Z(t) = 0$ 恰好等价于 $\zeta(\frac{1}{2} + \mathrm{i}t) = 0$。于是，*$Z$ 变号的地方就有临界线上的零点。*

4.2 一次完整的存在性认证

球算术给出以下严格界限（附录程序逐项检查）：

$$-0.000019941389 < Z(14.1347) < -0.000019941387 < 0,$$
$$0 < 0.000059374978 < Z(14.1348) < 0.000059374981.$$

两端严格异号，由介值定理，存在 $\gamma \in (14.1347, 14.1348)$ 使 $Z(\gamma) = 0$，即 $\frac{1}{2} + \mathrm{i}\gamma$ 是 ζ 的零点。这证明了一个临界线零点的存在，而不只是“图上看到曲线过了横轴”。

注意，计算得到的复球的虚部包含 0，这本身不能证明 Z 取实值；实值性来自函数方程。计算与论证各自承担的任务不能混淆。

4.3 为什么还不能说找全了

一次变号只保证区间里至少一个零点。同号的区间里也可能藏着两个零点，或一个偶重零点；而在临界线上取样，根本看不到临界线外的零点。所以，无论找到多少个变号区间，得到的都只是零点个数的下界。要得到“全部”，需要另一种信息。

本节的分工 论证：Z 实值且与 ζ 同零点；介值定理。 计算：两点处 Z 的符号。 核查：两个区间端点的符号严格确认。

5 完整性：Turing 方法怎样证明没有遗漏

5.1 思路

数零点有两种办法，它们看到的东西不同：

- **变号计数**只看临界线，给出临界线上零点个数的下界 $m(T)$；

- **辐角原理**给出临界带中零点的总数 $N(T)$，不论零点在不在临界线上。

如果 $m(T)=N(T)$，那么全部零点都已在临界线上找到，$\mathrm{RH}(T)$ 成立。困难在于，$N(T)$ 的公式中含有一项 $S(T)$，它没有可以直接严格计算的简单表达。Turing 的想法是：不去算 S 在某一点的值，而利用一个已知事实——S 的积分平均很小。如果漏掉了零点，计数差额会在其后一直存在，使相应的积分偏离到这个界限之外。下面把这个想法写成严格的判据。

5.2 把“少数了几个”写成一个函数

令 $N(t)$ 表示满足 $0<\operatorname{Re}\rho<1$、$0<\operatorname{Im}\rho\le t$ 的零点总数，按重数计，不预先假定 RH。在 t 不是零点纵坐标时，零点计数恒等式（本书第 11 章，由辐角原理得到）是

$$N(t)=1+\vartheta(t)/\pi+S(t),\qquad S(t)=\pi^{-1}\arg\zeta(\tfrac12+\mathrm{i}t),$$

辐角从 $\zeta(2)>0$ 出发，沿 $2\to 2+\mathrm{i}t\to\frac12+\mathrm{i}t$ 连续选取。在零点纵坐标处把 S 取为右极限，使它与 N 的右连续约定一致；这些离散点上的取值不影响积分。

设已找到一批区间 $[a_j,b_j]$，每个区间两端的 Z 严格异号，区间内部两两不交；相邻区间可以共用一个非零端点。定义

$$m(t)=\#\{j:b_j\le t\},\qquad \widetilde{S}(t)=m(t)-1-\vartheta(t)/\pi,$$

即把恒等式中的 N 换成已找到的个数 m。$m(t)$ 只在区间右端点到达后才计入这次发现，因而始终 $m(t)\le N(t)$。记

$$D(t)=N(t)-m(t)=S(t)-\widetilde{S}(t)\ge 0.$$

$D(t)$ 是非负整数，表示“少数了几个”。它既可能记录真正漏掉的零点，也可能记录“零点已经过去、所在区间的右端点还没到”的暂时差额，因此 D 并非处处单调。

5.3 遗漏会留下持续的差额

取认证高度 T，要求它不落在任何变号区间的内部。对 $t\ge T$，每个在 $(T,t]$ 内新计入的区间，都在 $[T,t]$ 内贡献至少一个零点；端点处 $Z\neq 0$，这些贡献互不重复。因此

$$N(t)-N(T)\ge m(t)-m(T),\qquad \text{即}\qquad D(t)\ge D(T)\quad(t\ge T).$$

于是，若高度 T 以下漏掉了至少一个零点（$D(T)\ge 1$），则 $D\ge 1$ 在 $[T,\infty)$ 上一直成立，从而

$$\int_T^{T'} D(t)\,\mathrm{d}t\ge T'-T.$$

另一方面，解析数论给出 S 的积分不能太大。本文使用 Trudgian 的结果 [3, 定理 2.2]：

$$\left|\int_T^{T'} S(t)\,\mathrm{d}t\right|\le U(T'):=2.067+0.059\log T',\qquad T'>T>168\pi.$$

它对全部零点成立，不以 RH 为前提；下面只用它的上界。两者一比较：遗漏造成的偏移随 $T'-T$ 线性增长，而允许的偏移只按 $\log T'$ 增长，延伸得足够远就会出现矛盾。

命题 5.1 (排除遗漏的判据)**.** 设 $T'>T>168\pi$，且 T 不在任何变号区间内部。如果严格算出一个下界

$$L\le\int_T^{T'}\widetilde{S}(t)\,\mathrm{d}t,\qquad \text{并且}\qquad L+(T'-T)>U(T'),$$

那么 $N(T)=m(T)$。

证明. 若 $D(T)\ge 1$，则

$$\int_T^{T'} S(t)\,\mathrm{d}t=\int_T^{T'}\widetilde{S}(t)\,\mathrm{d}t+\int_T^{T'} D(t)\,\mathrm{d}t\ge L+(T'-T)>U(T'),$$

与积分上界矛盾。所以 $D(T)=0$，即 $N(T)=m(T)$。 □

此时每个已计入的变号区间至少含一个临界线零点，而全部零点按重数计只有 $m(T)$ 个。故每个区间恰有一个单零点，再无其他零点，尤其没有临界线外的零点。这就证明了 RH(T)。零点都是单零点是这个判据附带给出的结论；RH 本身并不要求单零点。

5.4 判据中的积分怎样计算

未知的 S 不必计算，需要的只是由已知端点构成的 $\widetilde{S}$ 的积分。$m(t)$ 是阶梯函数：在 T 处已有 $m(T)$ 次计数，其后每到一个右端点 b_j 增加 1，所以

$$\int_T^{T'} m(t)\,\mathrm{d}t=m(T)(T'-T)+\sum_{T<b_j\le T'}(T'-b_j),$$

端点取有理数时可以精确计算。记

$$J=\int_T^{T'}\frac{\vartheta(t)}{\pi}\,\mathrm{d}t,\qquad Q=\int_T^{T'}\widetilde{S}(t)\,\mathrm{d}t=\int_T^{T'} m(t)\,\mathrm{d}t-(T'-T)-J,$$

于是只需 J 的严格上界，就得到 Q 的下界 L。

求 J 的界不需要高阶求积公式。把 $[T,T']$ 分成小区间 $X_k=[u_k,v_k]$，若球算术给出 $\vartheta(X_k)\subset[\ell_k,r_k]$，则

$$(v_k-u_k)\ell_k\le\int_{u_k}^{v_k}\vartheta(t)\,\mathrm{d}t\le(v_k-u_k)r_k,$$

逐段相加再除以 π 即得 J 的包含区间。这里送入函数的是整个小区间，而不是只取中点，因此没有遗漏求积余项。

5.5 实际算完一例：认证 RH(600)

取 $T=600$、$T'=610$，满足 $600>168\pi\approx 527.8$，可以使用上面的积分界。

第一步：变号计数 在格点 $t_k=k/20$（$k=0,\dots,12200$）上严格确认 $Z(t_k)$ 的符号，记录相邻异号的区间。$T=600$ 是格点，不在任何区间内部。结果是

$$m(600)=341,\qquad m(610)=348,$$

600 以上新增的七个右端点为 601.65, 602.60, 603.65, 604.65, 606.40, 608.45, 609.40。到这一步，341 还只是“找到的个数”。

第二步：两个积分 由求和公式，

$$\int_{600}^{610} m(t)\,\mathrm{d}t=341\times 10+\sum_j(610-b_j)=3443.2.$$

把 $[600,610]$ 分成 1000 个宽 0.01 的小区间做区间求积，得到 $3433.6<J<3433.8$。所以

$$Q=3443.2-10-J>-0.6,\qquad \text{取}L=-0.6.$$

第三步：判据 右端向外取整，$U(610) = 2.067 + 0.059 \log 610 < 2.446$。于是

$$\boxed{L + (610 - 600) = 9.4 > 2.446 > U(610).}$$

由命题 5.1，$N(600) = 341$：高度 600 以下全部非平凡零点都在临界线上，而且都是单零点。注意，搜索到 610 是为了认证到 600，这次判据没有认证到 610。

附录程序在 160 位和 224 位二进制精度下各完整运行一次，全部符号判定和不等式均通过。重算是为了复核；每一次运行能用于证明，依据仍是球算术的包含保证与上述定理，而不是两次结果相同。

5.6 判据不通过时怎么办

若某个函数值区间包含 0，程序不能猜测符号，应提高精度或换一个采样点。若判别不等式不成立，就不能声称找全；原因可能是采样太疏漏掉了变号、区间过宽、积分估计过粗，也可能确有未找到的零点。特别是，不能想当然地认为 $\widetilde{S}$ 的积分“总在零附近”：由于 $\widetilde{S} = S - D$，遗漏会使它持续向下偏。本节给出的是一个*充分判据*，不是对任何输入都必然成功的算法。

本节的分工 论证：零点计数恒等式；S 的积分界（Trudgian）；命题 5.1。 计算：格点上 Z 的符号；J 的区间求积。 核查：最终只需核对一个不等式 $9.4 > 2.446$，以及它所依赖的每一项严格界。

6 形式化核查又多检查了什么

上面的认证把推导、参数和程序都写明了，读者可以重做每一步。但它不是由证明助手核验过的形式化证明。它的可信度依赖于：所引定理（零点计数恒等式、Trudgian 的积分界）的正确性；Arb 函数实现的包含保证；程序是否正确实现了采样和求积；以及编译器、程序库版本和硬件等运行环境。重复运行和独立实现能增加核查力度，但不能代替数学上的误差论证。

在 Lean 等证明助手中，命题用精确定义写出，证明组织为形式系统中的推导，由一个很小的内核逐步检查 [6]。以第 2 节为例：形式化时要写出“素数”的定义，再证明 $641 \times 6700417 = 2^{32} + 1$，最终得到“$2^{32} + 1$ 不是素数”的形式证明。因子可以由外部程序找到，但它只是待核对的数据；不能把外部程序返回的“是/否”直接当作定理。

若要形式化第 5.5 节，还要处理零点计数恒等式、积分界、区间算法的正确性，以及计算与这些定理之间的连接；仅把程序放在证明助手旁边运行，并没有完成这些工作。形式化也不取消全部信任：仍要核对形式化命题是否表达了原意。例如“高度 600 以下所有零点在临界线上”与“找到的 341 个零点在临界线上”是两个不同的命题；后者即使有完全正确的形式证明，也不能替代前者。

7 有限计算能走多远

本例的逻辑是：有限次符号检查和严格求积，加上对全部零点成立的解析定理，推出 RH(600)。Platt–Trudgian 用同样的逻辑（严格数值计算加 Turing 型计数）认证了 RH($3 \cdot 10^{12}$)[4]；规模大得多，结论仍有高度限制。

有限计算也可以成为无穷范围命题证明的一部分，前提是另有数学论证覆盖其余的无穷部分。例如 Helfgott 对三素数定理（每个大于 5 的奇数都是三个素数之和）的证明：解析论证处理足够大的奇数，有限计算验证到 8.875×10^{30} 为止的范围 [5]，两部分合起来覆盖全部奇数。

对 RH 而言，逐段提高的有限高度认证不能代替关于所有高度的证明；计算机能做的，是帮助寻找或核查一个适用于所有高度的一般论证。

8 小结

读一份计算证明，可以依次问五个问题。本文四个例子的回答如下：

问题	F_5 是合数	$\lambda_1 > 0$	$(14.1347, 14.1348)$ 内有零点	RH(600)
证明哪个命题?	有非平凡因子	一个实数为正	临界线上存在零点	全部零点在线上
检查哪些有限事实?	一次乘法	两个常数的区间	两个符号	12201 个符号与 1000 段求积
误差怎样包住?	无误差	区间端点	球算术	球算术
范围有无遗漏?	不涉及	不涉及	只断言“至少一个”	Turing 判据排除遗漏
谁来核查?	任何人手算	有理数比较	附录程序	附录程序；最后一行可手算

四个例子的共同结构是：**定理把无穷多的可能归结为有限个检查，计算可靠地完成这些检查，核查保证检查本身可以重复**。

A 可直接复核的程序

下面是本篇使用的完整 Python 程序，依次核对第 2、3 节的精确算式、λ_1 的区间界、第 4 节的变号，以及第 5.5 节的计数与求积。保存为 `认证示例.py`，安装 `python-flint==0.9.0` 后用 `python3 认证示例.py` 运行；不要使用会跳过 `assert` 的 `-O` 选项。

`arb` 表示实球，`acb` 表示复球。小数从字符串构造，使输入区间包含所写的精确十进制数；格点用整数除法构造，不经过普通浮点数。函数 `sign` 只有在整个实球位于 0 的一侧时才返回符号，否则立即停止。求积中的 `arb(0, arb(1)/200)` 是以 0 为中心、半径 $1/200$ 的球，因此 `X` 包含整个小区间 $[k/100, (k+1)/100]$，累加的不是中点处的单个函数值。

```
"""Reproduce every check in note 16.
Install: python3 -m pip install python-flint==0.9.0
Run without -O: python3 认证示例.py
"""
import flint
from flint import arb, acb, ctx
from fractions import Fraction


def exact_checks():
    # Section 2: the certificate 641 * 6700417 for F_5 = 2^32 + 1
    assert 2**32 + 1 == 4294967297
    assert 641 * 6700417 == 2**32 + 1
    # Section 3: rational bounds for sqrt(2)
    assert Fraction("1.414")**2 < 2 < Fraction("1.415")**2


def theta(t):
    return (acb(arb(1)/4, t/2).lgamma().imag
            - t*arb.pi().log()/2)


def Z(t):
    w = acb(0, theta(t)).exp()*acb(arb(1)/2, t).zeta()
    assert w.imag.contains(0)
    return w.real


def sign(x):
    if x > 0:
        return 1
    if x < 0:
        return -1
    raise ValueError("Sign not certified; stop.")


def local_checks():
    # Section 3: lambda_1 > 0
    e = arb.const_euler()
```

```
    g = (4*arb.pi()).log()
    assert arb("0.57721") < e < arb("0.57722")
    assert arb("2.53102") < g < arb("2.53103")
    lam = 1 + e/2 - g/2
    assert arb("0.02309") < lam < arb("0.02310")
    # Section 4: a sign change of Z near the first zero
    assert arb("-0.000019941389") < Z(arb("14.1347"))
    assert Z(arb("14.1347")) < arb("-0.000019941387")
    assert arb("0.000059374978") < Z(arb("14.1348"))
    assert Z(arb("14.1348")) < arb("0.000059374981")


def count_and_integrate():
    # Section 5: Turing's criterion for RH(600)
    assert arb(600) > 168*arb.pi()
    right = []
    previous = sign(Z(arb(0)))
    for k in range(1, 12201):
        current = sign(Z(arb(k)/20))
        if current != previous:
            right.append(k)
        previous = current
    m = sum(k <= 12000 for k in right)
    tail = [k for k in right if k > 12000]
    assert m == 341
    assert tail == [12033, 12052, 12073, 12093,
                    12128, 12169, 12188]
    area = Fraction(10*m) + sum(
        (Fraction(12200-k, 20) for k in tail), Fraction(0))
    assert area == Fraction(17216, 5)

    J = arb(0)
    for k in range(60000, 61000):
        X = arb(2*k+1)/200 + arb(0, arb(1)/200)
        J += theta(X)/100
    J /= arb.pi()
    assert arb("3433.6") < J < arb("3433.8")
    A = arb(area.numerator)/area.denominator
    Q = A - 10 - J
    L = arb("-0.6")
    U = arb("2.067") + arb("0.059")*arb(610).log()
    assert Q > L
    assert U < arb("2.446")
    assert L + 10 > U
    print(ctx.prec, "bits: m(600) =", m,
          "; m(610) =", len(right))
    print("J in (3433.6, 3433.8); Q > -0.6; U < 2.446")
    print("Turing test passed: 9.4 > 2.446")


```

```
if __name__ == "__main__":
    exact_checks()
    print("python-flint", flint.__version__)
    for bits in (160, 224):
        with ctx.workprec(bits):
            local_checks()
            count_and_integrate()
```

两次运行均应得到 $m(600) = 341$、$m(610) = 348$，并显示 `Turing test passed: 9.4 > 2.446`。这些输出是检查通过后的摘要；说明它们为什么足以支持结论的，是程序中的逐项断言和正文中的数学论证。

References

[1] V. R. Pratt, Every prime has a succinct certificate, *SIAM Journal on Computing* 4 (1975), 214–220.

[2] F. Johansson, Arb: efficient arbitrary-precision midpoint-radius interval arithmetic, *IEEE Transactions on Computers* 66 (2017), 1281–1292. https://arxiv.org/abs/1611.02831.

[3] T. Trudgian, Improvements to Turing's method, *Mathematics of Computation* 80 (2011), 2259–2279, 尤其定理 2.2. https://arxiv.org/abs/0903.1885.

[4] D. Platt and T. Trudgian, The Riemann hypothesis is true up to $3 \cdot 10^{12}$, *Bulletin of the London Mathematical Society* 53 (2021), 792–797. https://arxiv.org/abs/2004.09765.

[5] H. A. Helfgott and D. J. Platt, Numerical verification of the ternary Goldbach conjecture up to $8.875 \cdot 10^{30}$, *Experimental Mathematics* 22 (2013), 406–409. https://arxiv.org/abs/1305.3062.

[6] *The Lean Language Reference*, Introduction 与 Validating a Lean Proof. https://lean-lang.org/doc/reference/latest/.